COGNITHOR AUTONOMOUS EVOLUTION ENGINE — IMPLEMENTATION SPEC

You are implementing a major architectural extension to the Cognithor codebase. This document is your complete specification. Read it entirely before writing any code. Do not skip sections. Do not simplify. Implement everything described here with production-grade quality.

TABLE OF CONTENTS

1. Project Context & Goals
2. Entry Points (start_cognithor.bat / install.sh)
3. System Detector (Hardware & Environment Discovery)
4. Operating Mode Selection (Local / Hybrid / Cloud)
5. Multi-Agent Architecture (Orchestrator / Scout / Skill-Builder)
6. Idle Loop Engine (Curiosity-Driven Autonomous Learning)
7. Resource Monitor & Cooperative Scheduling
8. State Persistence & Interrupt/Resume
9. API Key Management & Security
10. Configuration Schema
11. File Structure
12. Error Handling & Edge Cases
13. Testing Requirements
14. Implementation Order

---

1. PROJECT CONTEXT & GOALS

Cognithor is a local-first autonomous agent OS powered by Ollama/LM Studio. The existing codebase already has: CLI, WebUI, Telegram, Discord, and other channels; a 5-tier cognitive memory system; an Obsidian-compatible Knowledge Vault; a Reflector that evaluates sessions; and a config.yaml-driven setup.

What We Are Adding

We are adding an **Autonomous Evolution Engine** — a system that makes Cognithor actively learn, research, and build new skills *during idle time* when no user is interacting with it. The system must:

- **Detect hardware at startup** and recommend the optimal operating mode
- **Support three operating modes**: pure local, hybrid (local + API), pure cloud
- **Run three specialized agents** with different roles and resource profiles
- **Execute an idle learning loop** that discovers knowledge gaps, researches solutions, and builds new skills autonomously
- **Respect system resources** — instantly yield to user interaction and other processes
- **Persist state** so interrupted loops resume exactly where they stopped
- **Remain fully portable** — the same codebase must work on a high-end workstation, a laptop, and a cloud server

Design Principles

- **Portability first**: No hardcoded hardware assumptions. Everything adapts at runtime.
- **User always wins**: Any user interaction immediately preempts all idle activity.
- **Graceful degradation**: If GPU is unavailable, fall back to CPU. If CPU is overloaded, pause. If internet is down, skip web research and do local tasks only.
- **Atomic operations**: Every idle loop step must be completable in a bounded time and checkpointable at its boundary.
- **Security by default**: Sandboxed skill execution, no auto-registration without approval, encrypted API key storage.
- **Budget awareness**: Every mode has configurable daily limits for API calls, web searches, and skill builds.

---

2. ENTRY POINTS

2.1 install.sh (Linux/macOS)

Location: `./install.sh`

```
#!/bin/bash
# Cognithor Installation & First-Run Bootstrap
```

This script must:

1. Check for Python >= 3.10, install if missing via system package manager (detect apt/dnf/brew)
2. Create a virtual environment in `.venv/` if not present
3. Install all Python dependencies from `requirements.txt`
4. Check for Ollama — if not installed, print instructions but do NOT auto-install (respect user choice)
5. Check for LM Studio — same: detect, inform, don't force
6. Run the System Detector (Section 3) for the first time
7. Run the Mode Selection wizard (Section 4) interactively in the terminal
8. Generate initial `~/.cognithor/config.yaml`
9. If hybrid or cloud mode is selected, prompt for API key(s) and store them (Section 9)
10. Print a summary and instructions for starting Cognithor
11. Create a convenience launcher script at `./start_cognithor.sh`

The script must be idempotent — running it again should detect existing installation and offer to reconfigure or update.

2.2 Start_cognithor.bat (Windows)

Location: `./Start_cognithor.bat`

```
@echo off
REM Cognithor Launcher for Windows
```

This script must:

1. Check if Python is available in PATH; if not, check common install locations (AppData, Program Files)
2. If `.venv/` does not exist, run the setup wizard (equivalent to install.sh)
3. Activate the virtual environment
4. Run the main entry point: `python -m cognithor.main`
5. If the setup wizard has never been run, trigger first-run bootstrap
6. Keep the terminal window open on error (pause)

2.3 start_cognithor.sh (Linux/macOS launcher, generated by install.sh)

```
#!/bin/bash
cd "$(dirname "$0")"
source .venv/bin/activate
python -m cognithor.main "$@"
```

2.4 Main Entry Point: cognithor/main.py

On every startup, the main entry point must:

1. Load existing config from `~/.cognithor/config.yaml` (or create default)
2. Run the System Detector (fast mode — cached results, re-scan only if hardware changed)
3. Validate that the configured mode still works with detected hardware
4. If validation fails (e.g., GPU was removed, Ollama uninstalled), trigger interactive re-configuration
5. Initialize the agents according to the configured mode
6. Start the Resource Monitor background thread
7. Start the Idle Detector
8. Begin listening for user input on all configured channels
9. Print startup banner with system info summary

---

3. SYSTEM DETECTOR

3.1 Module: cognithor/system/detector.py

The System Detector gathers all hardware and software information needed to determine which operating modes are available.

3.2 Detection Targets

Each detection target produces a `DetectionResult`:

```
@dataclass
class DetectionResult:
    key: str            # e.g., "gpu"
    value: str          # Human-readable description
    status: str         # "ok" | "warn" | "fail"
    raw_data: dict      # Machine-readable details
```

Implement these detectors:

3.2.1 Operating System

- Detect OS name, version, architecture (platform module)
- Detect if running in Docker/WSL/VM (check for /.dockerenv, /proc/version for WSL, systemd-detect-virt)
- `raw_data`: `{"os": "linux", "distro": "ubuntu", "version": "24.04", "arch": "x86_64", "is_wsl": false, "is_docker": false}`

3.2.2 CPU

- Core count (physical and logical), model name, max frequency
- Use `psutil.cpu_count()`, `psutil.cpu_freq()`, and parse `/proc/cpuinfo` on Linux or `wmic` on Windows
- Status: “ok” if >= 4 physical cores, “warn” if 2-3, “fail” if 1
- `raw_data`: `{"model": "AMD Ryzen 9 9950X3D", "physical_cores": 16, "logical_cores": 32, "max_freq_mhz": 5700}`

3.2.3 RAM

- Total, available, percentage used
- Use `psutil.virtual_memory()`

- Status: “ok” if >= 16GB total, “warn” if 8-16GB, “fail” if < 8GB
- `raw_data`: `{"total_gb": 256, "available_gb": 240, "percent_used": 6.2}`

3.2.4 GPU

- Detect NVIDIA GPUs via `nvidia-smi` (subprocess call, parse XML output with `--query-gpu=name,memory.total,memory.free,driver_version --format=csv`)
- Detect AMD GPUs via `rocm-smi` if available
- Detect Apple Silicon via `sysctl` (macOS) — check for Metal support
- Detect Intel Arc via `xpu-smi` if available
- If no dedicated GPU: report integrated graphics
- Status: “ok” if dedicated GPU with >= 8GB VRAM, “warn” if 4-8GB, “fail” if none or < 4GB
- `raw_data`: `{"vendor": "nvidia", "model": "RTX 5090", "vram_total_gb": 32, "vram_free_gb": 30, "driver": "570.1", "cuda_version": "12.8"}`

3.2.5 Ollama Detection

- Check if Ollama is running: HTTP GET to `http://localhost:11434/api/tags`
- If not running, check if binary exists (`which ollama` or check common install paths)
- If running, enumerate all available models
- Status: “ok” if running with models, “warn” if installed but not running, “fail” if not found
- `raw_data`: `{"installed": true, "running": true, "endpoint": "http://localhost:11434", "models": [{"name": "qwen2.5:32b", "size_gb": 18.5, "parameter_size": "32B", "quantization": "Q4_K_M"}]}`

3.2.6 LM Studio Detection

- Check if LM Studio API is running: HTTP GET to `http://localhost:1234/v1/models`
- Same pattern as Ollama: installed vs running vs models available
- `raw_data`: `{"installed": true, "running": true, "endpoint": "http://localhost:1234", "models": [...]}`

3.2.7 Disk Space

- Check free space on the partition where `~/.cognithor/` resides
- Status: “ok” if >= 50GB free, “warn” if 10-50GB, “fail” if < 10GB
- `raw_data`: `{"path": "/home", "total_gb": 1000, "free_gb": 847}`

3.2.8 Network

- Check internet connectivity: try DNS resolution + HTTPS GET to a reliable endpoint (e.g., https://api.anthropic.com, https://api.openai.com)
- Optionally measure approximate bandwidth (download a small file, measure time)
- Check if specific API endpoints are reachable
- Status: “ok” if internet available, “warn” if slow (< 5 Mbit/s), “fail” if no connectivity
- `raw_data`: `{"internet": true, "bandwidth_mbps": 120, "reachable_apis": ["anthropic", "openai", "google"]}`

3.3 Detection Runner

```
class SystemDetector:
    def run_full_scan(self) -> SystemProfile:
        """Run all detectors, return complete profile."""

    def run_quick_scan(self) -> SystemProfile:
        """Use cached results for stable items (OS, CPU, RAM, disk).
        Re-scan volatile items (GPU state, Ollama running, network)."""

    def save_profile(self, path: str):
        """Save to ~/.cognithor/system_profile.json"""

    def load_cached_profile(self) -> Optional[SystemProfile]:
        """Load cached profile, return None if stale (>24h) or missing."""
```

The `SystemProfile` must include a method:

```
def get_available_modes(self) -> list[str]:
    """Return list of available modes: ['local', 'hybrid', 'cloud']"""

def get_recommended_mode(self) -> str:
    """Return the single best mode for this hardware."""

def get_optimal_agent_config(self, mode: str) -> dict:
    """Return the best agent configuration for the given mode on this hardware.
```

```
    This includes: which model for each agent, which device (gpu/cpu/cloud),
    what context length, what batch size, etc."""
```

3.4 Model Selection Logic

When local models are available, the detector must choose the optimal assignment:

```
IF mode == "local" or mode == "hybrid":
    - Orchestrator gets the BEST local model (largest, on GPU)
    - Scout gets the SMALLEST local model (on CPU via llama.cpp CPU inference)
    - Skill-Builder:
        - In "local" mode: same as Orchestrator (queued, async)
        - In "hybrid" mode: API model (claude-sonnet-4 preferred)

IF mode == "cloud":
    - All three agents use API models
    - Orchestrator: best available (claude-sonnet-4 or gpt-4o)
    - Scout: cheapest fast model (claude-haiku or gpt-4o-mini)
    - Skill-Builder: same as Orchestrator
```

For local model selection, rank by: parameter count > quantization quality > context length. Prefer coding-specialized models for Skill-Builder when available (e.g., qwen2.5-coder over generic qwen2.5).

3.5 Terminal Output During Detection

Display a clean, real-time progress view:

```
> cognithor --bootstrap
  System Detection...

  🖥️  OS          ✓  Linux 6.8 (Ubuntu 24.04)
  ⚙️  CPU         ✓  AMD Ryzen 9 9950X3D · 16C/32T
  💾  RAM         ✓  256 GB DDR5
  🎮  GPU         ✓  NVIDIA RTX 5090 · 32 GB GDDR7
  🦙  Ollama      ✓  v0.9.2 running · 3 models
  🧠  Models      ✓  qwen2.5-32b-coder, llama3.1-8b, nomic-embed
  💾  Disk        ✓  847 GB free
  🌐  Network     ✓  Internet available · 120 Mbit/s

  Recommended mode: ⚡ Hybrid
```

Use the `rich` library for formatting (already a Cognithor dependency).

---

4. OPERATING MODE SELECTION

4.1 Mode Definitions

LOCAL Mode

- All inference runs on local hardware
- No internet required for operation (only for web research during idle loop)
- Zero API costs
- Privacy: maximum — all data stays on machine
- Agents: Orchestrator (GPU), Scout (CPU), Skill-Builder (GPU, queued)
- Idle Loop budget: unlimited cycles, limited only by hardware

HYBRID Mode

- Daily operations run locally
- Skill-Builder agent uses API for complex code generation
- Requires internet + API key
- Low API costs (~$0.50-2.00/day depending on activity)
- Privacy: high — only skill-building prompts leave the machine
- Agents: Orchestrator (GPU), Scout (CPU), Skill-Builder (API)
- Idle Loop budget: configurable API calls per day

CLOUD Mode

- All inference via API
- Requires internet + API key
- Higher API costs (~$5-20/day depending on activity)
- Privacy: moderate — all prompts go to API provider
- Works on any machine (even a Raspberry Pi)
- Agents: all three via API (different model tiers)
- Idle Loop budget: strictly configurable API calls per day

4.2 Mode Selection Wizard

Interactive terminal UI (use `rich` + `questionary` or `InquirerPy`):

```
┌──────────────────────────────────────────┐
│  COGNITHOR — Operating Mode Selection    │
└──────────────────────────────────────────┘

Based on your system, these modes are available:

  [1] 🏠  Local     Everything on your hardware. No cloud.
  [2] ⚡  Hybrid    Local + API for skill building. (RECOMMENDED)
  [3] ☁️   Cloud     Everything via API. For light hardware.

  Your choice [1/2/3]: _
```

If only one mode is available (e.g., no GPU and no internet), skip the wizard and auto-select with an explanation.

After selection:

- If hybrid or cloud: prompt for API provider and key (Section 9)
- Generate config.yaml
- Print summary of agent assignments

4.3 Runtime Mode Validation

On every startup, validate:

1. If mode is “local” or “hybrid”: Is the assigned GPU still available? Is Ollama/LM Studio running? Are the configured models present?
2. If mode is “hybrid” or “cloud”: Is internet available? Is the API key still valid? (Quick auth check)
3. If validation fails: Print a clear message explaining what changed, and offer to reconfigure or fall back to a compatible mode.

---

5. MULTI-AGENT ARCHITECTURE

5.1 Agent Base Class

```
# cognithor/agents/base.py

class Agent:
    name: str
    role: str
    model: str
    device: str           # "gpu", "cpu", "cloud"
    priority: int         # 0 = highest (user), 3 = lowest (idle housekeeping)
    max_context: int

    async def invoke(self, prompt: str, system: str = None, **kwargs) -> str:
        """Send a prompt to this agent's model and return the response."""

    async def invoke_structured(self, prompt: str, schema: dict, **kwargs) -> dic
        """Invoke and parse response as structured JSON."""

    def is_available(self) -> bool:
        """Check if this agent can currently accept work."""

    def estimated_cost(self, input_tokens: int, output_tokens: int) -> float:
        """Return estimated cost in USD. Returns 0.0 for local models."""
```

5.2 Orchestrator Agent

File: `cognithor/agents/orchestrator.py`

- **Role**: Primary brain. Handles all user interactions, makes decisions, coordinates other agents.
- **Model**: Best available local model (in local/hybrid mode) or best API model (cloud mode)
- **Device**: GPU (local/hybrid) or Cloud
- **Priority**: 0 (always gets resources first)
- **Capabilities**:
  - Processes user messages from all channels
  - Decides whether to handle a task itself or delegate to Scout/Skill-Builder
  - Manages the idle loop state machine
  - Evaluates Scout findings and decides on actions
  - Reviews Skill-Builder output before registration

5.3 Scout Agent

File: `cognithor/agents/scout.py`

- **Role**: Lightweight worker. Handles background tasks that don't need the best model.
- **Model**: Smallest available local model (local/hybrid) or cheapest API model (cloud)
- **Device**: CPU (local/hybrid) — CRITICAL: must never touch the GPU
- **Priority**: 3 (lowest — always yields)
- **Capabilities**:
  - Log analysis and pattern detection
  - Web search and result summarization
  - Knowledge vault indexing and link maintenance
  - Quality audits on existing knowledge entries
  - Simple text classification and tagging

For CPU-only inference with Ollama, use:

```
CUDA_VISIBLE_DEVICES="" ollama run llama3.1:8b
```

Or set `num_gpu: 0` in the Ollama modelfile to force CPU-only execution.

For LM Studio, configure the model to use CPU-only in the LM Studio settings.

5.4 Skill-Builder Agent

File: `cognithor/agents/skill_builder.py`

- **Role**: Specialist for complex code generation. Only invoked when a new skill needs to be built.
- **Model**: API model (hybrid mode) or best local model (local mode) or best API model (cloud mode)
- **Device**: Cloud (hybrid) or GPU-queued (local) or Cloud (cloud)
- **Priority**: 2 (above Scout, below Orchestrator)
- **Capabilities**:
  - Receives a structured skill specification from the Orchestrator

- Generates complete, tested Python code
- Includes docstrings, type hints, error handling
- Returns code + test cases + documentation
- Each invocation is a self-contained task (no multi-turn conversations)

Skill-Builder Request Format

```
@dataclass
class SkillBuildRequest:
    name: str                    # e.g., "calculate_insurance_premium"
    description: str             # What the skill should do
    input_schema: dict           # Expected inputs (JSON Schema)
    output_schema: dict          # Expected outputs (JSON Schema)
    context: str                 # Domain context and constraints
    examples: list[dict]         # Input/output examples
    dependencies_allowed: list   # Allowed pip packages
    max_execution_time: int      # Seconds
```

Skill-Builder Response Format

```
@dataclass
class SkillBuildResponse:
    code: str                    # The complete Python skill code
    test_code: str               # pytest-compatible test code
    documentation: str           # Markdown documentation
    dependencies: list[str]      # Required pip packages
    confidence: float            # Self-assessed confidence 0.0-1.0
    notes: str                   # Any caveats or limitations
```

5.5 Agent Communication

Agents communicate through an async message queue:

```
# cognithor/agents/message_bus.py

class MessageBus:
    async def send(self, from_agent: str, to_agent: str, message: AgentMessage):
        """Send a message to another agent."""

    async def receive(self, agent_name: str, timeout: float = None) -> AgentMessa
        """Wait for next message for this agent."""
```

```
    async def broadcast(self, from_agent: str, message: AgentMessage):
        """Send to all agents."""

@dataclass
class AgentMessage:
    type: str           # "task", "result", "status", "interrupt", "shutdown"
    payload: dict
    priority: int
    timestamp: float
    correlation_id: str  # Track request-response pairs
```

---

6. IDLE LOOP ENGINE

6.1 Module: cognithor/idle_loop/engine.py

The Idle Loop is the core of the autonomous evolution system. It runs when no user is interacting with Cognithor.

6.2 Idle Detection

```
class IdleDetector:
    def __init__(self, threshold_minutes: float = 5.0):
        self.last_user_activity: float = time.time()
        self.threshold = threshold_minutes * 60

    def record_activity(self):
        """Called on every user interaction."""
        self.last_user_activity = time.time()

    @property
    def is_idle(self) -> bool:
        return (time.time() - self.last_user_activity) > self.threshold

    @property
    def idle_duration(self) -> float:
        return max(0, time.time() - self.last_user_activity - self.threshold)
```

Additional idle signals to detect (implement where supported):

- Screen saver active (D-Bus on Linux, Windows API)
- No mouse/keyboard input (optional, requires OS-level hooks)

- System load average below threshold
- Configured quiet hours (e.g., always idle between 2-6 AM)

6.3 The Five-Phase Loop

The idle loop executes five phases in sequence. Each phase is an atomic unit — it completes fully or checkpoints its progress before yielding.

Phase 1: Self-Inventory (~30 seconds)

Agent: **Scout** (CPU)

Tasks:

1. Enumerate all registered skills with last-used dates and success/failure rates
2. Scan recent conversation logs for: topics that came up frequently, tasks that failed or had low confidence, questions the system couldn't answer
3. Analyze the Knowledge Vault for: stale entries (> 30 days old), orphaned entries (no links), gaps in topic coverage
4. Produce a structured inventory report

Output format:

```
@dataclass
class InventoryReport:
    total_skills: int
    unused_skills: list[str]          # Skills not used in > 30 days
    failed_tasks: list[FailedTask]     # Recent failures with context
    frequent_topics: list[str]         # Topics that came up often
    knowledge_gaps: list[str]          # Identified missing areas
    stale_knowledge: list[str]         # Entries needing refresh
    vault_stats: dict                  # Size, link density, coverage
```

Phase 2: Gap Analysis (~1 minute)

Agent: **Orchestrator** (GPU) — only if idle, otherwise **Scout** (CPU)

Tasks:

1. Receive the InventoryReport from Phase 1
2. Cross-reference with the user profile and configured focus areas

3. Rank gaps by estimated impact: “How much would filling this gap improve the system for the user?”
4. Select the single highest-priority gap to work on this cycle
5. Generate a specific action plan: what to research, what skill to build, what knowledge to acquire

Output format:

```
@dataclass
class GapAnalysis:
    selected_gap: str
    impact_score: float            # 0.0 - 1.0
    action_type: str               # "research" | "skill_build" | "knowledge_refr
    action_plan: str               # Detailed instructions for next phases
    search_queries: list[str]      # Pre-generated search queries
    skill_spec: Optional[SkillBuildRequest]  # If action_type is skill_build
```

Phase 3: Research (~3 minutes)

Agent: **Scout** (CPU) for web search, **Orchestrator** (GPU) for evaluation — only if idle

Tasks:

1. Execute web searches using the queries from Phase 2
2. Filter results for quality: relevance, recency, source trustworthiness
3. Fetch and summarize relevant pages
4. Extract actionable information: API endpoints, code examples, best practices, data sources
5. Cross-reference findings with existing knowledge to avoid duplicates
6. Save raw research to a temporary staging area

Quality filter criteria:

- Source trust score >= configured threshold (default 0.7)
- Content must be from the last 2 years (configurable)
- Exclude SEO spam, affiliate content, AI-generated filler
- Prefer: official documentation, academic papers, reputable tech blogs, GitHub repos with > 100 stars

Output format:

```
@dataclass
class ResearchReport:
    topic: str
    sources: list[Source]
    summary: str
    key_findings: list[str]
    relevant_apis: list[APIReference]
    relevant_code: list[CodeSnippet]
    quality_score: float
    suggested_next_steps: list[str]
```

Phase 4: Skill Building (~5 minutes)

Agent: **Skill-Builder** (API in hybrid mode, GPU in local mode)

This phase only runs if the action_type from Phase 2 is "skill_build" or "skill_improve". Otherwise, skip to Phase 5.

Tasks:

1. Orchestrator formulates a precise SkillBuildRequest using research from Phase 3
2. Skill-Builder generates the code, tests, and documentation
3. Orchestrator reviews the response for obvious issues
4. Code is executed in the sandbox (see Section 6.5)
5. If tests pass: skill is staged for approval
6. If tests fail: error is logged, skill is saved as draft for next cycle
7. If `auto_register_skills` is true in config: register automatically
8. If false: save to `~/.cognithor/vault/skill_drafts/` with a notification for the user

Sandbox execution rules (CRITICAL):

- Execute in a subprocess with resource limits (max CPU time, max memory, no network access)
- Use `resource` module on Linux, `psutil` process limits on Windows
- Whitelist of allowed imports (configurable, default: standard library + requests, json, re, datetime, math, csv, pathlib)

- Timeout: configurable, default 60 seconds
- No file system access outside of a designated temp directory
- Capture stdout, stderr, and return code

Phase 5: Knowledge Integration (~1 minute)

Agent: **Scout** (CPU)

Tasks:

1. Take all new knowledge from this cycle (research summaries, skill documentation, findings)
2. Format as Obsidian-compatible markdown with YAML frontmatter
3. Save to the appropriate vault folder (recherchen/, wissen/, skills/)
4. Generate backlinks to related existing entries
5. Update the vault search index
6. Write a cycle completion log entry
7. Update the idle loop statistics (total cycles, skills built, knowledge added, etc.)
8. Determine if another cycle should run (check: still idle? resources available? budget remaining?)

6.4 Cycle Orchestration

```
# cognithor/idle_loop/engine.py

class IdleLoopEngine:
    def __init__(self, config, agents, resource_monitor, state_manager):
        self.config = config
        self.agents = agents
        self.resource_monitor = resource_monitor
        self.state = state_manager
        self.current_phase: int = 0
        self.is_running: bool = False

    async def run(self):
        """Main loop — runs indefinitely while idle."""
        while True:
            # Wait for idle condition
            await self.wait_for_idle()
```

```
        # Check budget
        if not self.budget_remaining():
            await self.sleep_until_budget_reset()
            continue

        # Check resources
        zone = self.resource_monitor.current_zone
        if zone == ResourceZone.PAUSE:
            await self.sleep_until_resources_free()
            continue

        # Resume from checkpoint if available
        checkpoint = self.state.load_checkpoint()
        start_phase = checkpoint.phase if checkpoint else 1

        # Execute phases
        try:
            for phase in range(start_phase, 6):
                # Check for interrupts before each phase
                if self.should_interrupt():
                    self.state.save_checkpoint(phase, self.current_data)
                    break

                self.current_phase = phase
                await self.execute_phase(phase)

                # Checkpoint after each phase
                self.state.save_checkpoint(phase + 1, self.current_data)

            # Cycle complete
            self.state.clear_checkpoint()
            self.stats.record_cycle_complete()

        except Exception as e:
            self.state.save_checkpoint(self.current_phase, self.current_data)
            self.log_error(e)
            await asyncio.sleep(60)  # Back off on error

    def should_interrupt(self) -> bool:
        """Check all interrupt conditions."""
        return (
            not self.idle_detector.is_idle or
            self.resource_monitor.current_zone == ResourceZone.PAUSE or
            self.shutdown_requested
        )
```

6.5 Sandbox Environment

```
# cognithor/idle_loop/sandbox.py

class SkillSandbox:
    def __init__(self, config):
        self.timeout = config.get("timeout_seconds", 60)
        self.max_memory_mb = config.get("max_memory_mb", 512)
        self.allowed_imports = config.get("allowed_imports", [...])
        self.temp_dir = tempfile.mkdtemp(prefix="cognithor_sandbox_")

    async def execute(self, code: str, test_code: str) -> SandboxResult:
        """Execute skill code and its tests in an isolated environment."""
        # 1. Write code to temp file
        # 2. Static analysis: check for forbidden imports, os.system calls, etc.
        # 3. Execute in subprocess with resource limits
        # 4. Capture output and return code
        # 5. Clean up temp directory

    def static_analysis(self, code: str) -> list[str]:
        """Check for dangerous patterns before execution.
        Returns list of warnings/errors."""
        # Check for: os.system, subprocess, eval, exec, __import__
        # Check for: file operations outside temp_dir
        # Check for: network access (socket, urllib, requests in non-allowed cont
        # Check for: infinite loops (basic heuristic: while True without break)
```

---

7. RESOURCE MONITOR & COOPERATIVE SCHEDULING

7.1 Module: cognithor/system/resource_monitor.py

The Resource Monitor runs as a background thread and continuously tracks system resource usage.

7.2 Resource Zones

```
class ResourceZone(Enum):
    FREE = "free"           # All resources available — full speed
    THROTTLED = "throttled" # Resources getting tight — reduce activity
    PAUSE = "pause"         # Resources critical — stop all idle work
```

Zone thresholds (all configurable):

```
FREE:       CPU < 30% AND RAM < 60% AND GPU_VRAM < 50%
THROTTLED:  CPU < 70% AND RAM < 80% AND GPU_VRAM < 80%
PAUSE:      CPU >= 70% OR RAM >= 80% OR GPU_VRAM >= 80% OR user_active
```

7.3 Monitoring Implementation

```
class ResourceMonitor:
    def __init__(self, config):
        self.check_interval = config.get("check_interval_seconds", 5)
        self.history_window = config.get("history_window_seconds", 30)
        self.cpu_history: deque = deque(maxlen=self.history_window // self.check_
        self.current_zone: ResourceZone = ResourceZone.PAUSE  # Start conservativ
        self._running = False

    async def start(self):
        """Start background monitoring."""
        self._running = True
        while self._running:
            snapshot = self.take_snapshot()
            self.cpu_history.append(snapshot.cpu_percent)
            self.current_zone = self.calculate_zone(snapshot)

            # Emit event on zone change
            if self.current_zone != self._previous_zone:
                await self.emit_zone_change(self._previous_zone, self.current_zon

            await asyncio.sleep(self.check_interval)

    def take_snapshot(self) -> ResourceSnapshot:
        return ResourceSnapshot(
            cpu_percent=psutil.cpu_percent(interval=0.1),
            cpu_avg=statistics.mean(self.cpu_history) if self.cpu_history else 0,
            ram_percent=psutil.virtual_memory().percent,
            ram_available_gb=psutil.virtual_memory().available / (1024**3),
            gpu_vram_percent=self._get_gpu_usage(),
            gpu_vram_free_gb=self._get_gpu_free_vram(),
            disk_free_gb=shutil.disk_usage(str(Path.home())).free / (1024**3),
        )

    def _get_gpu_usage(self) -> float:
        """Get GPU VRAM usage percentage. Returns 0 if no GPU."""
        try:
            result = subprocess.run(
                ["nvidia-smi", "--query-gpu=memory.used,memory.total", "--format=
                capture_output=True, text=True, timeout=5
```

```
            )
            used, total = map(float, result.stdout.strip().split(","))
            return (used / total) * 100
        except Exception:
            return 0.0
```

7.4 GPU Inference Scheduling

CRITICAL: When the Orchestrator needs the GPU for a user request, the Idle Loop must release it IMMEDIATELY.

```
class InferenceScheduler:
    def __init__(self):
        self._gpu_lock = asyncio.Lock()
        self._user_waiting = asyncio.Event()

    async def request_gpu(self, priority: int) -> AsyncContextManager:
        """Request GPU access. Higher priority preempts lower."""
        if priority == 0:  # User request
            self._user_waiting.set()
            # The idle loop checks _user_waiting before each inference call
            # and immediately releases the GPU

    async def idle_inference(self, agent, prompt, **kwargs):
        """Run inference for idle loop. Yields immediately if user needs GPU."""
        if self._user_waiting.is_set():
            return None  # Abort, user needs GPU

        async with self._gpu_lock:
            if self._user_waiting.is_set():
                return None  # Double-check after acquiring lock
            result = await agent.invoke(prompt, **kwargs)
            return result
```

7.5 Priority System

```
Priority 0: User Interaction      — ALWAYS gets resources immediately
Priority 1: System Processes      — OS, Ollama server, other apps
Priority 2: Idle Loop (Skill-Builder)  — Runs only when GPU free
Priority 3: Idle Loop (Scout tasks)    — Runs on CPU, very low impact
Priority 4: Idle Loop (Housekeeping)   — Vault indexing, log cleanup
```

---

8. STATE PERSISTENCE & INTERRUPT/RESUME

8.1 Module: cognithor/idle_loop/state_manager.py

Every phase boundary is a checkpoint. The state manager ensures zero data loss on interrupts.

8.2 Checkpoint Format

```
@dataclass
class IdleLoopCheckpoint:
    cycle_id: str                    # UUID for this cycle
    phase: int                       # Next phase to execute (1-5)
    started_at: float                # Timestamp when cycle started
    interrupted_at: Optional[float]  # Timestamp of last interrupt
    resume_count: int                # How many times this cycle was resumed

    # Phase-specific data
    inventory_report: Optional[dict]  # Output of Phase 1
    gap_analysis: Optional[dict]      # Output of Phase 2
    research_report: Optional[dict]   # Output of Phase 3
    skill_build_result: Optional[dict]  # Output of Phase 4

    # Research progress (Phase 3 can be partially complete)
    search_queries_completed: list[str]
    search_queries_remaining: list[str]
    research_artifacts: list[dict]    # Partially collected research
```

8.3 State Storage

```
class StateManager:
    CHECKPOINT_PATH = Path.home() / ".cognithor" / "idle_state.json"
    HISTORY_PATH = Path.home() / ".cognithor" / "idle_history.jsonl"

    def save_checkpoint(self, phase: int, data: dict):
        """Atomically save checkpoint. Uses write-to-temp-then-rename pattern."""
        checkpoint = IdleLoopCheckpoint(phase=phase, **data)
        temp_path = self.CHECKPOINT_PATH.with_suffix(".tmp")
        temp_path.write_text(json.dumps(asdict(checkpoint), default=str))
        temp_path.rename(self.CHECKPOINT_PATH)  # Atomic on POSIX

    def load_checkpoint(self) -> Optional[IdleLoopCheckpoint]:
        """Load existing checkpoint, return None if no checkpoint."""
        if not self.CHECKPOINT_PATH.exists():
```

```
            return None
        try:
            data = json.loads(self.CHECKPOINT_PATH.read_text())
            return IdleLoopCheckpoint(**data)
        except (json.JSONDecodeError, TypeError):
            self.CHECKPOINT_PATH.unlink()  # Corrupted, start fresh
            return None

    def clear_checkpoint(self):
        """Clear checkpoint after successful cycle completion."""
        if self.CHECKPOINT_PATH.exists():
            self.CHECKPOINT_PATH.unlink()

    def record_history(self, cycle_summary: dict):
        """Append cycle summary to history log."""
        with open(self.HISTORY_PATH, "a") as f:
            f.write(json.dumps(cycle_summary, default=str) + "\n")
```

8.4 Interrupt Handling

```
# Register signal handlers for graceful shutdown
import signal

def setup_signal_handlers(engine: IdleLoopEngine):
    def handle_interrupt(signum, frame):
        engine.request_shutdown()
        # Engine will save checkpoint at next phase boundary

    signal.signal(signal.SIGINT, handle_interrupt)
    signal.signal(signal.SIGTERM, handle_interrupt)

    # On Windows, also handle CTRL_CLOSE_EVENT
    if sys.platform == "win32":
        import win32api
        win32api.SetConsoleCtrlHandler(lambda x: engine.request_shutdown(), True)
```

---

9. API KEY MANAGEMENT & SECURITY

9.1 Storage

API keys are stored in `~/.cognithor/.env`:

```
COGNITHOR_ANTHROPIC_KEY=sk-ant-...
COGNITHOR_OPENAI_KEY=sk-...
COGNITHOR_GOOGLE_KEY=AIza...
COGNITHOR_GROQ_KEY=gsk_...
```

File permissions: `chmod 600 ~/.cognithor/.env` (owner read/write only). On Windows: Use `icacls` to restrict access to the current user.

9.2 Loading

```
# cognithor/system/secrets.py

class SecretManager:
    ENV_PATH = Path.home() / ".cognithor" / ".env"

    def __init__(self):
        self._secrets = {}
        self._load()

    def _load(self):
        """Load from .env file using python-dotenv."""
        if self.ENV_PATH.exists():
            from dotenv import dotenv_values
            self._secrets = dotenv_values(self.ENV_PATH)

    def get(self, key: str) -> Optional[str]:
        """Get a secret. Checks .env file, then environment variables."""
        return self._secrets.get(key) or os.environ.get(key)

    def set(self, key: str, value: str):
        """Save a secret to the .env file."""
        self._secrets[key] = value
        self._write()
        self._set_permissions()

    def _write(self):
        self.ENV_PATH.parent.mkdir(parents=True, exist_ok=True)
        lines = [f"{k}={v}" for k, v in self._secrets.items()]
        self.ENV_PATH.write_text("\n".join(lines) + "\n")

    def _set_permissions(self):
        if sys.platform != "win32":
            os.chmod(self.ENV_PATH, 0o600)
        else:
            # Windows: restrict to current user
```

```
            subprocess.run(
                ["icacls", str(self.ENV_PATH), "/inheritance:r",
                 "/grant:r", f"{os.environ['USERNAME']}:F"],
                capture_output=True
            )

    def validate_key(self, provider: str, key: str) -> bool:
        """Quick validation — make a minimal API call to check if key works."""
        # Implement per-provider: Anthropic, OpenAI, Google, etc.
```

9.3 .gitignore

Ensure these are in `.gitignore`:

```
~/.cognithor/.env
~/.cognithor/idle_state.json
~/.cognithor/system_profile.json
*.env
.env
```

Also add to the Cognithor repo's `.gitignore` if not already present.

---

10. CONFIGURATION SCHEMA

10.1 Full config.yaml Schema

Location: `~/.cognithor/config.yaml`

```
# ═══════════════════════════════════════════════════
# COGNITHOR CONFIGURATION
# Generated by system bootstrap — safe to edit manually
# ═══════════════════════════════════════════════════

# Operating mode: "local" | "hybrid" | "cloud"
mode: "hybrid"

# System profile (auto-detected, do not edit unless you know what you're doing)
system:
  auto_detect: true
  # Override auto-detection:
  # force_gpu_device: "cuda:0"
  # force_cpu_threads: 8
```

```yaml
# ── Agent Configuration ──────────────────────────
agents:
  orchestrator:
    model: "ollama:qwen2.5-32b-coder"
    device: gpu
    priority: 0
    max_context_tokens: 32768
    temperature: 0.7

  scout:
    model: "ollama:llama3.1-8b"
    device: cpu
    priority: 3
    max_context_tokens: 8192
    temperature: 0.3
    cpu_threads: 4              # Limit CPU threads to avoid starving the system

  skill_builder:
    model: "api:claude-sonnet-4"
    device: cloud
    priority: 2
    max_context_tokens: 65536
    temperature: 0.2            # Low temp for reliable code generation

# ── API Configuration ────────────────────────────
api:
  # Keys are loaded from ~/.cognithor/.env
  # Set them there, not here!
  default_provider: "anthropic"

  providers:
    anthropic:
      env_key: "COGNITHOR_ANTHROPIC_KEY"
      models:
        default: "claude-sonnet-4-20250514"
        fast: "claude-haiku-4-5-20251001"
    openai:
      env_key: "COGNITHOR_OPENAI_KEY"
      models:
        default: "gpt-4o"
        fast: "gpt-4o-mini"
    google:
      env_key: "COGNITHOR_GOOGLE_KEY"
      models:
        default: "gemini-2.5-pro"
        fast: "gemini-2.5-flash"
```

```yaml
# ── Idle Loop Configuration ──────────────────────
idle_loop:
  enabled: true
  idle_threshold_minutes: 5
  max_cycle_duration_minutes: 15

  # Resource thresholds
  resources:
    cpu_max_percent: 30
    ram_max_percent: 60
    gpu_vram_max_percent: 50
    check_interval_seconds: 5
    history_window_seconds: 30

  # Daily budgets
  budget:
    max_api_calls_per_day: 50
    max_web_searches_per_day: 100
    max_skill_builds_per_day: 10
    max_cycles_per_day: 100
    budget_reset_hour: 0           # Reset at midnight

  # Quality control
  quality:
    min_source_trust_score: 0.7
    require_sandbox_test: true
    auto_register_skills: false    # If false, skills need human approval
    max_skill_complexity: "medium" # "simple" | "medium" | "complex"
    sandbox_timeout_seconds: 60
    sandbox_max_memory_mb: 512
    allowed_skill_imports:
      - "requests"
      - "json"
      - "re"
      - "datetime"
      - "math"
      - "csv"
      - "pathlib"
      - "collections"
      - "itertools"
      - "functools"
      - "typing"
      - "dataclasses"
      - "bs4"
      - "pandas"

  # Topic compass — guides what the idle loop researches
```

```
  focus_areas:
    - "insurance regulation Germany"
    - "sales optimization"
    - "customer communication"
    - "data analysis"
    - "Python automation"

  blocked_topics:
    - "cryptocurrency"
    - "political opinions"
    - "gambling"
    - "adult content"

  # Quiet hours — force idle loop even if user appears active
  quiet_hours:
    enabled: false
    start: "02:00"
    end: "06:00"

# ── Knowledge Vault ──────────────────────────────
vault:
  path: "~/.cognithor/vault/"
  folders:
    - "recherchen"
    - "wissen"
    - "skills"
    - "skill_drafts"
    - "meetings"
    - "projekte"
    - "daily"
  max_entry_age_days: 365        # Flag entries older than this for refresh
  auto_link: true                # Automatically generate backlinks

# ── Logging ──────────────────────────────────────
logging:
  level: "INFO"
  idle_loop_level: "DEBUG"       # More verbose for idle loop debugging
  log_dir: "~/.cognithor/logs/"
  max_log_size_mb: 50
  rotate_count: 5
```

10.2 Config Validation

On load, validate:

- Mode matches available hardware (quick scan)

- Referenced models exist (if local)
- API keys are present (if hybrid/cloud)
- All paths are writable
- Resource thresholds are sane (not contradictory)
- Budget values are positive integers

On validation failure: print clear error message with fix instructions. Do not crash — fall back to safe defaults where possible.

---

11. FILE STRUCTURE

Add these to the existing Cognithor project structure:

```
cognithor/
├── system/
│   ├── __init__.py
│   ├── detector.py              # Hardware & environment detection
│   ├── resource_monitor.py      # Real-time resource tracking
│   ├── secrets.py               # API key management
│   └── bootstrap.py             # First-run setup wizard
│
├── agents/
│   ├── __init__.py
│   ├── base.py                  # Agent base class
│   ├── orchestrator.py          # Main brain agent
│   ├── scout.py                 # Lightweight background worker
│   ├── skill_builder.py         # Code generation specialist
│   ├── message_bus.py           # Inter-agent communication
│   └── scheduler.py             # GPU/CPU scheduling & priorities
│
├── idle_loop/
│   ├── __init__.py
│   ├── engine.py                # Main idle loop orchestration
│   ├── idle_detector.py         # User activity detection
│   ├── state_manager.py         # Checkpoint persistence
│   ├── sandbox.py               # Isolated skill execution
│   ├── phases/
│   │   ├── __init__.py
│   │   ├── phase1_inventory.py  # Self-inventory
│   │   ├── phase2_analysis.py   # Gap analysis
│   │   ├── phase3_research.py   # Web research
```

```
│   │   ├── phase4_build.py      # Skill building
│   │   └── phase5_integrate.py  # Knowledge integration
│   └── quality/
│       ├── __init__.py
│       ├── source_scorer.py     # Web source trustworthiness
│       ├── skill_validator.py   # Skill quality checks
│       └── knowledge_auditor.py # Knowledge quality maintenance
│
├── config/
│   ├── __init__.py
│   ├── schema.py                # Config dataclasses & validation
│   └── defaults.py              # Default configuration values
│
└── main.py                      # Entry point (updated)

# Root level
install.sh                       # Linux/macOS installer
Start_cognithor.bat              # Windows launcher
```

---

12. ERROR HANDLING & EDGE CASES

Handle these scenarios explicitly:

12.1 Hardware Changes at Runtime

- GPU becomes unavailable (driver crash, external GPU disconnected): Switch Orchestrator to CPU mode, pause idle loop, notify user
- RAM pressure from other apps: Idle loop pauses, reduces model context lengths
- Ollama crashes/restarts: Detect via health check, reconnect automatically, retry failed requests

12.2 Network Issues

- Internet goes down during research phase: Save partial results, mark search queries as “pending”, resume when connectivity returns
- API rate limiting: Implement exponential backoff with jitter, reduce API calls per cycle
- API key invalidated: Notify user, disable API-dependent features, suggest reconfiguration

12.3 Idle Loop Edge Cases

- Cycle takes longer than max_cycle_duration_minutes: Force checkpoint and stop. Resume next idle period.
- Skill-Builder produces dangerous code: Static analysis catches it before sandbox execution. Log the attempt, do not execute.
- Knowledge Vault grows too large: Implement compaction — archive old entries, merge duplicates, remove low-quality entries.
- Budget exhausted mid-cycle: Complete current phase, checkpoint, stop until budget resets.
- Corrupted checkpoint file: Detect via JSON parse failure, delete checkpoint, start fresh cycle.

12.4 Multi-Platform Edge Cases

- Windows paths: Use pathlib.Path everywhere, never hardcode /
- macOS quarantine: Ollama may be quarantined on first run — detect and guide user
- Docker/containerized: Detect containerization, adjust GPU access (NVIDIA Container Toolkit), adjust disk paths
- WSL: Detect WSL, handle GPU passthrough correctly (WSL2 GPU support)

---

13. TESTING REQUIREMENTS

13.1 Unit Tests

For every module, write comprehensive unit tests. Minimum coverage: 85%.

Key test areas:

- System Detector: Mock all hardware calls, test all detection logic, test edge cases (no GPU, no network, partial Ollama, etc.)
- Resource Monitor: Test zone transitions, test history averaging, test concurrent access
- Idle Loop Engine: Test full cycle execution, test interrupt at every phase boundary, test checkpoint save/load
- Sandbox: Test code execution, test timeout handling, test forbidden import detection, test resource limits
- State Manager: Test atomic writes, test corruption recovery, test concurrent access

- Agent Communication: Test message passing, test priority handling, test timeout handling

13.2 Integration Tests

- Full bootstrap flow: detector → mode selection → config generation → agent initialization → idle loop start
- Interrupt/resume cycle: start idle loop → interrupt at phase 3 → verify checkpoint → resume → verify continuation
- Mode switching: start in hybrid → lose internet → automatic fallback to local → internet returns → switch back

13.3 Platform Tests

Run the test suite on:

- Ubuntu 24.04 (primary)
- Windows 11
- macOS (if available)
- WSL2

---

14. IMPLEMENTATION ORDER

Implement in this exact order. Each step must be fully complete and tested before moving to the next.

Step 1: System Detector

Build `cognithor/system/detector.py` with all 8 detection targets. Write tests. Verify on the current machine.

Step 2: Configuration Schema

Build `cognithor/config/schema.py` and `defaults.py`. Define all dataclasses, validation, and serialization.

Step 3: Secret Manager

Build `cognithor/system/secrets.py`. Implement .env storage with proper permissions.

Step 4: Bootstrap Wizard

Build `cognithor/system/bootstrap.py`. Interactive terminal UI for mode selection and initial setup.

Step 5: Entry Points

Build/update `install.sh`, `Start_cognithor.bat`, and `cognithor/main.py`. Ensure the full boot sequence works.

Step 6: Agent Base & Message Bus

Build `cognithor/agents/base.py`, `message_bus.py`, and `scheduler.py`.

Step 7: Three Agents

Build `orchestrator.py`, `scout.py`, `skill_builder.py`. Test each independently.

Step 8: Resource Monitor

Build `cognithor/system/resource_monitor.py`. Test zone transitions and GPU monitoring.

Step 9: Idle Loop Framework

Build `engine.py`, `idle_detector.py`, `state_manager.py`. Test the loop lifecycle without actual phase logic.

Step 10: Idle Loop Phases

Build all five phases one by one. Test each phase in isolation, then test the full cycle.

Step 11: Sandbox

Build `sandbox.py`. Test with safe and dangerous code samples.

Step 12: Quality Systems

Build source scorer, skill validator, and knowledge auditor.

Step 13: Integration Testing

Run full end-to-end tests. Fix everything that breaks.

Step 14: Documentation

Update the Cognithor README with the new features. Document all configuration options. Add examples.

FINAL NOTES

- NEVER break existing Cognithor functionality. All existing features (CLI, WebUI, Telegram, Discord, Memory, Vault, Reflector, etc.) must continue to work exactly as before.
- ALWAYS use async/await for I/O-bound operations.
- ALWAYS use type hints on all function signatures.
- ALWAYS write docstrings for public methods.
- ALWAYS handle exceptions explicitly — no bare `except:` clauses.
- ALWAYS use pathlib.Path for file operations.
- ALWAYS test on both Linux and Windows before considering a feature complete.
- The Idle Loop is the crown jewel of this implementation. It must be rock-solid, well-tested, and production-ready.
- Remember: This system will run unsupervised for hours. Every edge case matters. Every resource leak matters. Every unhandled exception matters. Build it like it needs to survive in the wild.

End of specification. Implement with precision.